

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

# SE 423: Introduction to Mechatronics

## Lecture 8-9: FFT and Filters

Marius Juston

Monday, February 16th, 2026

# Reminders

**Office Hours:**

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

**Lab:** Starting Lab 3 this week. This a 2-week lab!

**Homeworks:**

- Check-offs for homework 2 Ex 1 & 2 are due **February 17, 5PM (The end of office hours)**
- Check-offs for homework 2 Ex 3 & 4 are due **February 24, 5PM (The end of office hours)**
- Answers and code submissions for homework 2 are due **February 25, 9AM**

**LabVIEW:**

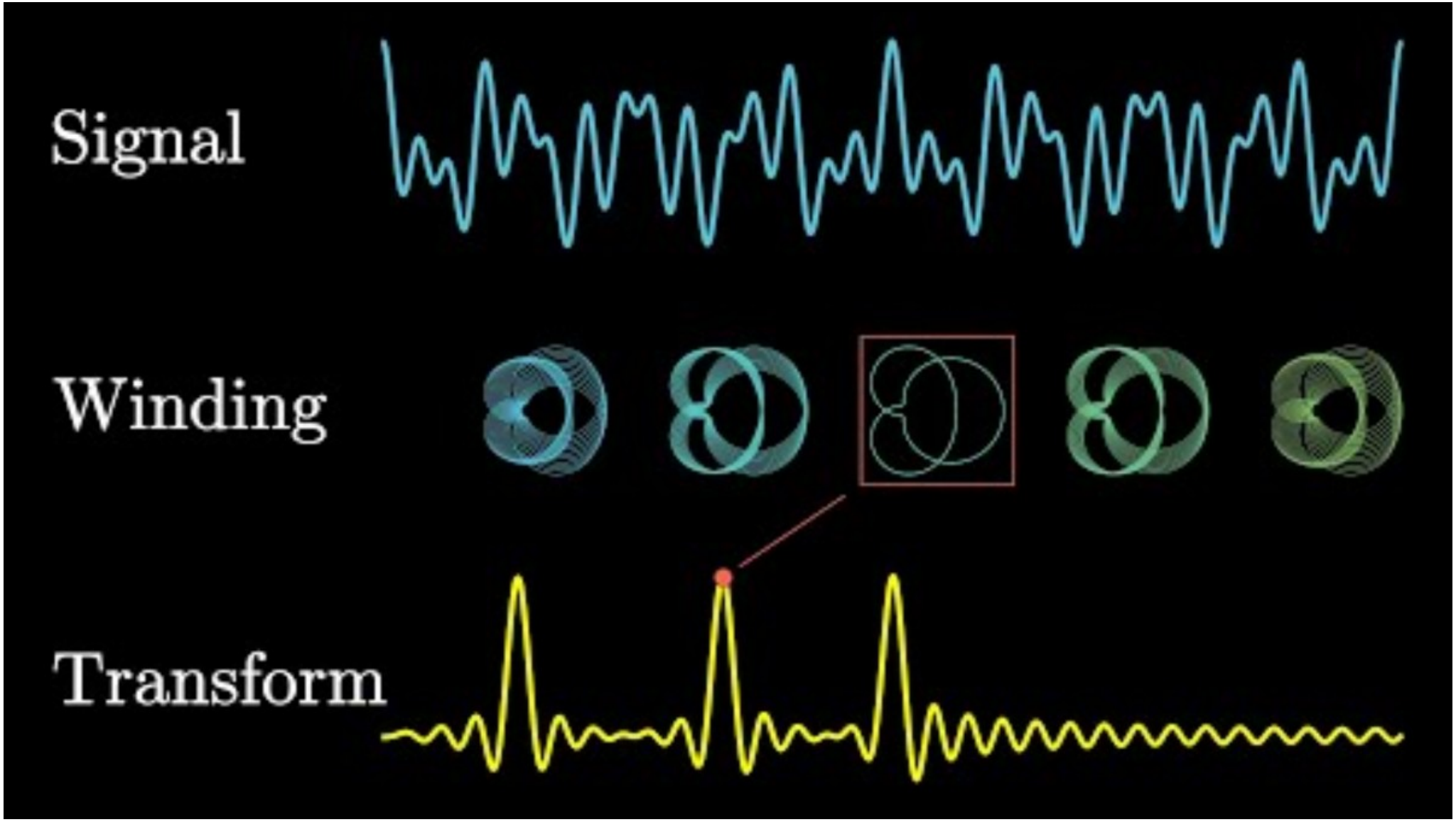
- All check-offs for LabVIEW 2 are due **February 26, 5PM**

# Questions? (1/2)

Homework, Lab, LabVIEW questions?

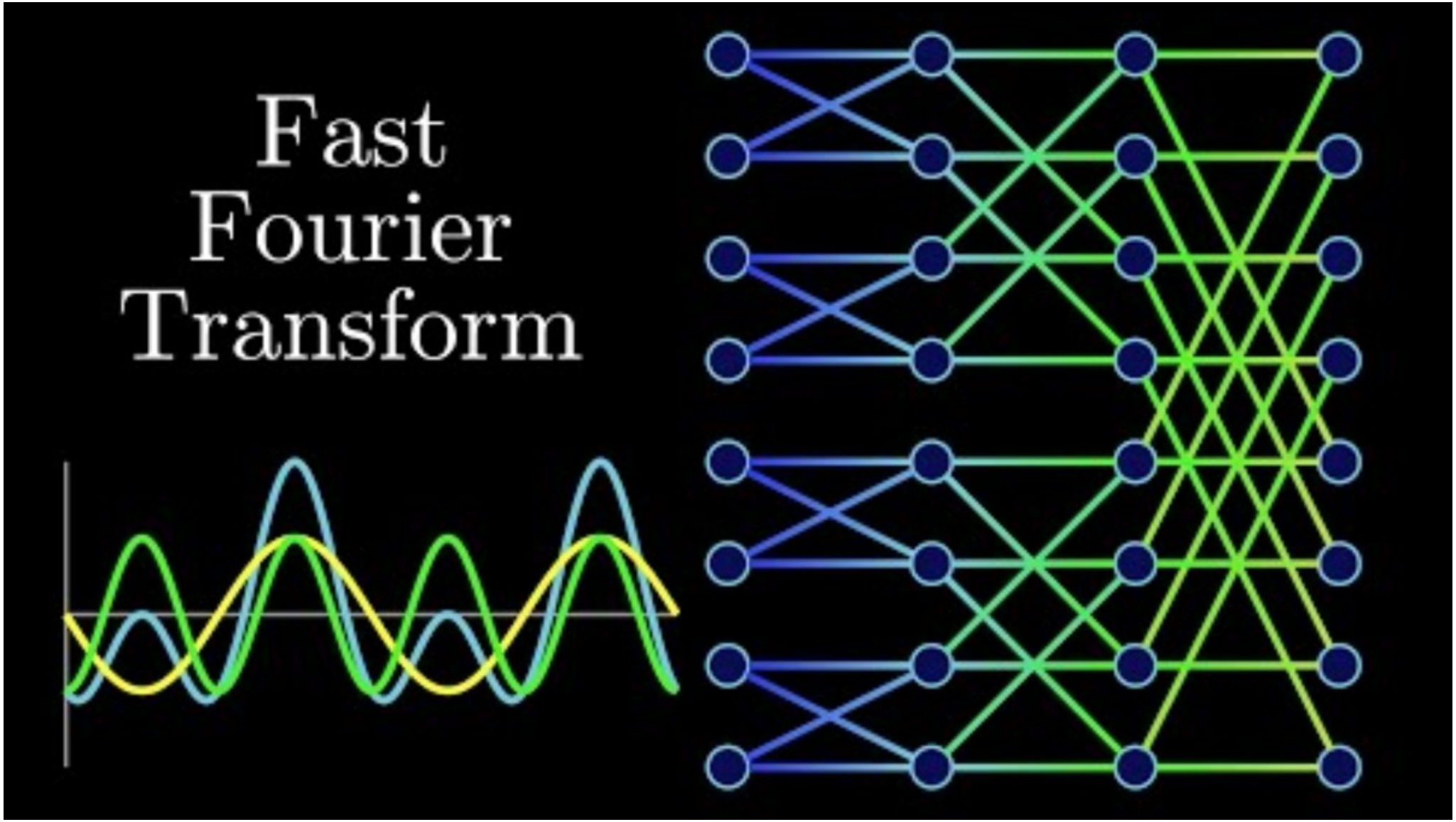
# Fourier Transforms & FFT Background

# Fourier Transforms




https://www.youtube.com/watch?v=spUNpyF58BY

# FFT




https://www.youtube.com/watch?v=h7apO7q16V0

# Fourier Transforms & FFT Code (1/2)

# FFT Code (1/5)

You just watched a couple of videos all about what a Fourier transform is, and what the FFT is...

Time to implement it!

Just joking, smart people have already implemented the algorithms and optimized it to work with this specific hardware to be as efficient as possible.

This one was implemented to work in C++ and C.

# FFT Code (2/5)

```
#define RFFT_STAGES     10
#define RFFT_SIZE       (1 << RFFT_STAGES)          // 1024

#pragma DATA_SECTION(pwrSpec, "FFT_buffer_2")
float pwrSpec[(RFFT_SIZE/2)+1];
float maxpwr = 0;
int16_t maxpwrindex = 0;

#pragma DATA_SECTION(test_output, "FFT_buffer_2")
float test_output[RFFT_SIZE];

#pragma DATA_SECTION(fft_input, "FFT_buffer_1")
float fft_input[RFFT_SIZE];

#pragma DATA_SECTION(RFFTF32Coef,"FFT_buffer_2")
float RFFTF32Coef[RFFT_SIZE];

#pragma DATA_SECTION(AdcPingBufRaw, "FFT_buffer_2")
uint16_t AdcPingBufRaw[RFFT_SIZE];

#pragma DATA_SECTION(AdcPongBufRaw, "FFT_buffer_2")
uint16_t AdcPongBufRaw[RFFT_SIZE];

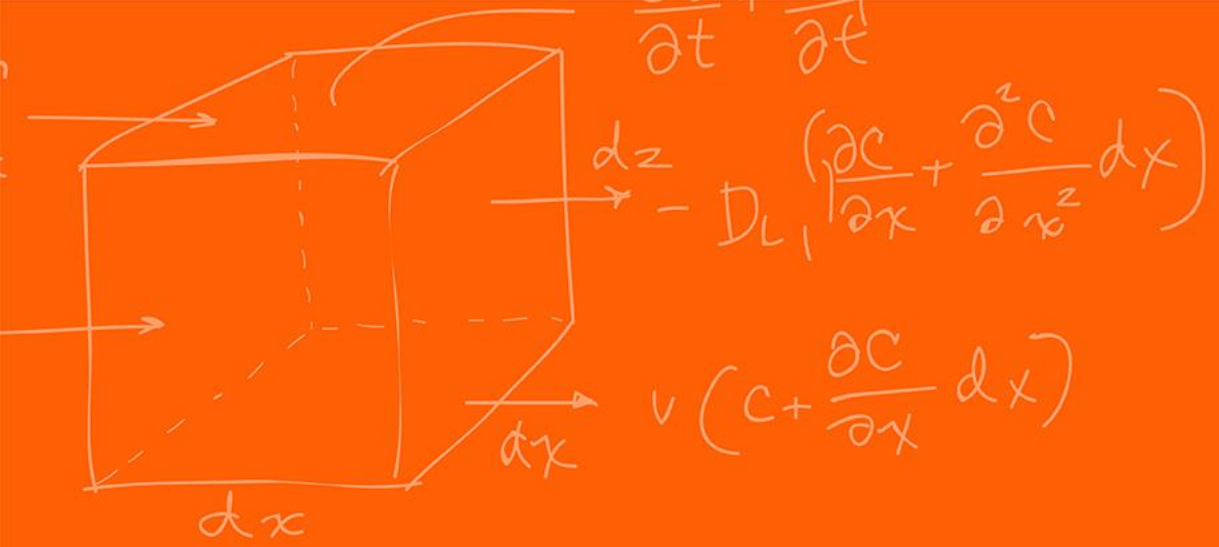
RFFT_F32_STRUCT rfft;
RFFT_F32_STRUCT_Handle hnd_rfft = &rfft;
uint16_t pingFFT = 0;
uint16_t pongFFT = 0;
uint16_t pingpongFFT = 1;
uint16_t iPingPong = 0;
int16_t DMAcount = 0;
__interrupt void DMA_ISR(void);
void InitDma(void);
```

```
typedef struct {
  float  *InBuf;        //!< Pointer to the input buffer
  float  *OutBuf;       //!< Pointer to the output buffer
  float  *CosSinBuf;    //!< Pointer to the twiddle factors
  float  *MagBuf;       //!< Pointer to the magnitude buffer
  float  *PhaseBuf;     //!< Pointer to the phase buffer
  uint16_t   FFTSize;   //!< Size of the FFT (number of real data
points)
  uint16_t   FFTStages; //!< Number of FFT stages
} RFFT_F32_STRUCT;

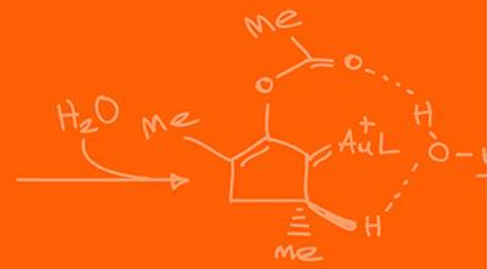

typedef RFFT_F32_STRUCT* RFFT_F32_STRUCT_Handle;
```

# FFT Code (3/5)

```
EALLOW;  // This is needed to write to EALLOW protected registers
PieVectTable.DMA_CH1_INT = &DMA_ISR;
EDIS;    // This is needed to disable write to EALLOW protected
registers
```

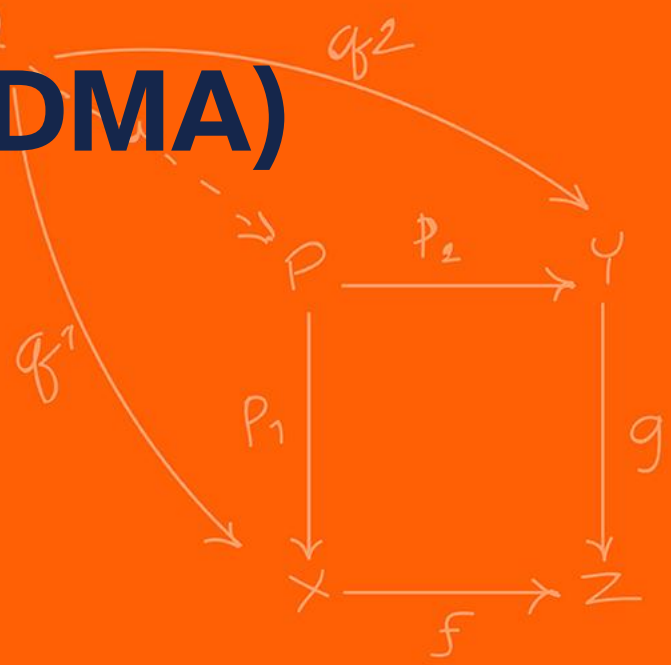








# Direct Memory Access (DMA)



# DMA (1/2)

The direct memory access (DMA) module provides a hardware method of transferring data between peripherals and/or memory without intervention from the CPU, thereby freeing up bandwidth for other system functions.

Many times, applications spend a significant amount of their bandwidth moving data,whether it is from off-chip memory to on-chip memory, or from a peripheral such as an analog-to-digital converter (ADC) to RAM, or even from one peripheral to another.

Furthermore, many times this data comes in a format that is not conducive to the optimum processing powers of the CPU.

The DMA module described in this reference guide can free up CPU bandwidth and rearrange the data into a pattern for more streamlined processing.

# DMA (2/2)

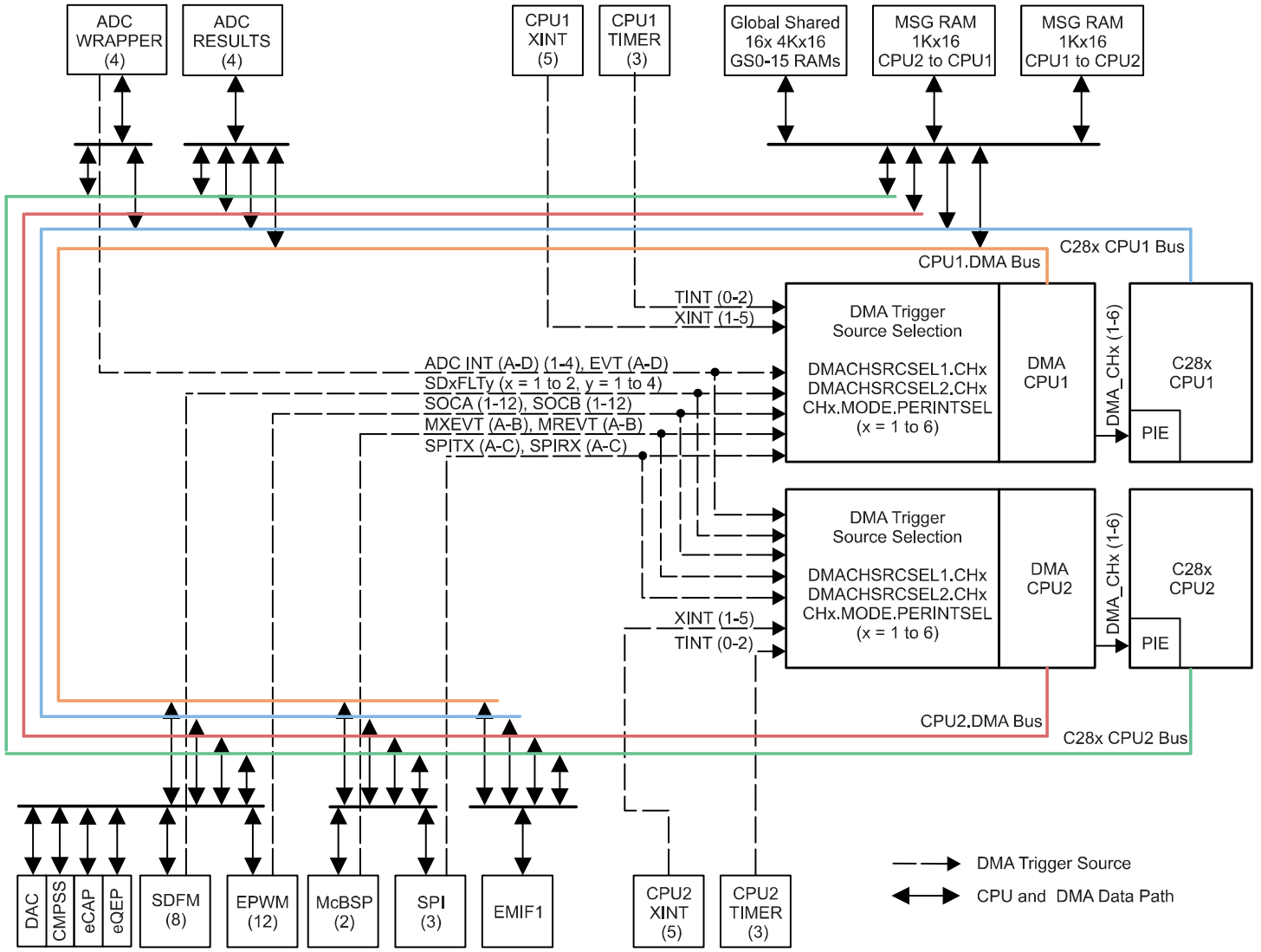
DMA features include:

- Six channels with independent PIE interrupts
- Each DMA channel can be triggered from multiple peripheral trigger sources independently
- Word Size: 16-bit or 32-bit (SPI limited to 16-bit)
- Throughput: 3 cycles/word without arbitration

The DMA has an 8-bit trigger source selection (“255 options”). The trigger sources range from the ADC, XINT, ePWM, Timer interrupts, SPI and more.

# FFT Code (4/5)

**Figure 5-1. DMA Block Diagram**



# FFT Code – Ping Pong Buffer (1/3)

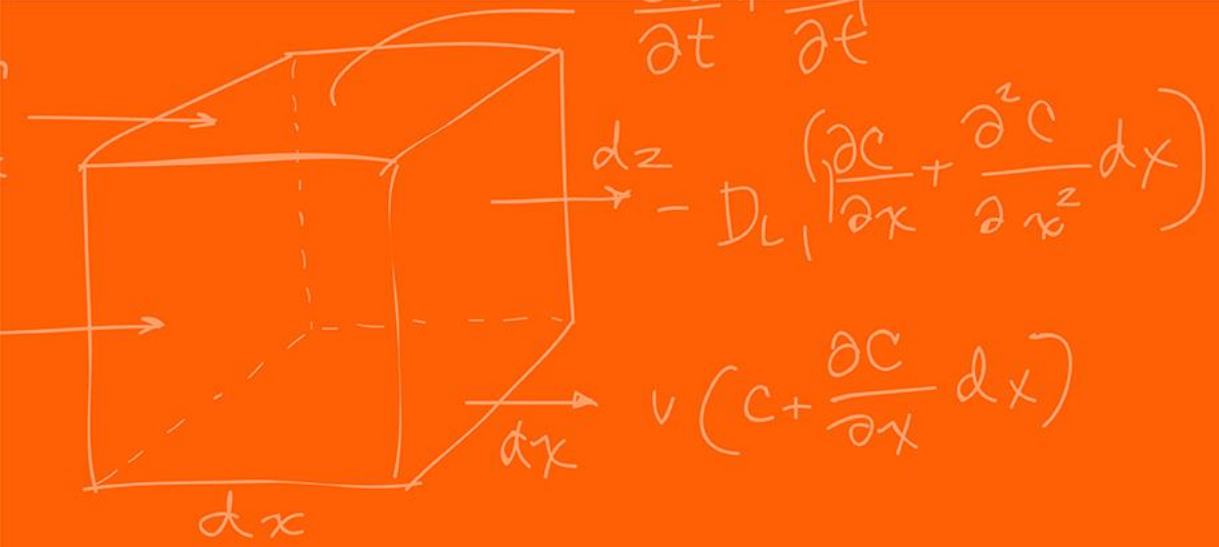
```
__interrupt void DMA_ISR(void)                      // PIE7.1 @ 0x000DA0  DMA channel 1 interrupt
{
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP7;         // Must acknowledge the PIE group

    //--- Process the ADC data
    if(iPingPong == 0)  // Ping buffer filling, process Pong buffer
    {
        // Manage the DMA registers
        EALLOW;                                                     // Enable EALLOW protected register access
        DmaRegs.CH1.DST_ADDR_SHADOW = (Uint32)AdcPongBufRaw;        // Adjust DST start address for Pong buffer
        EDIS;                                                       // Disable EALLOW protected register access

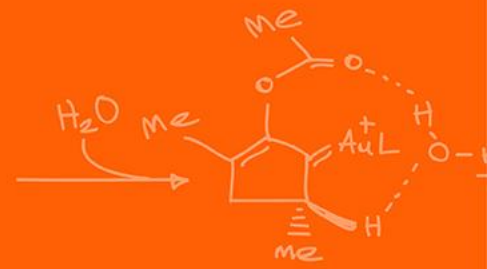
        // Don't run ping FFT first time around
        if (DMAcount > 0) {
            pingFFT = 1;
        }
        iPingPong = 1;
    }
    else    // Pong buffer filling, process Ping buffer
    {
        // Manage the DMA registers
        EALLOW;                                                     // Enable EALLOW protected register access
        DmaRegs.CH1.DST_ADDR_SHADOW = (Uint32)AdcPingBufRaw;        // Adjust DST start address for Ping buffer
        EDIS;                                                       // Disable EALLOW protected register access

        pongFFT = 1;
        iPingPong = 0;
    }

    DMAcount += 1;
}
```

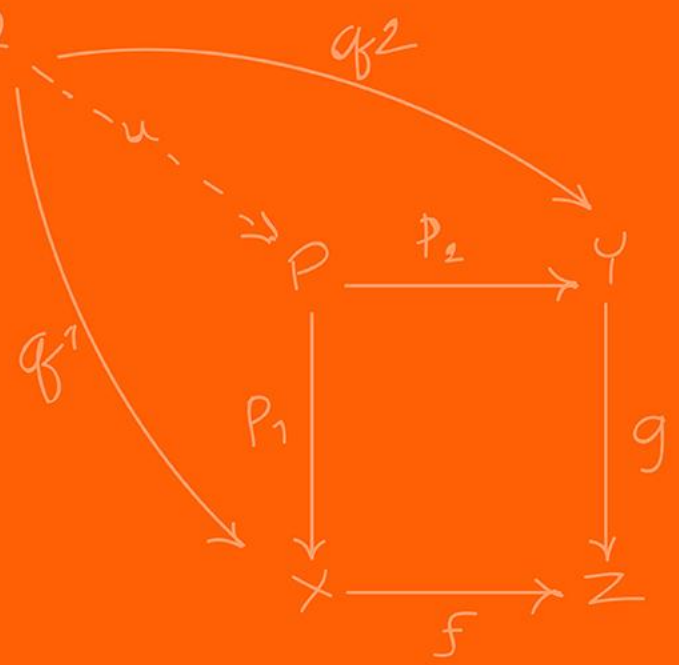




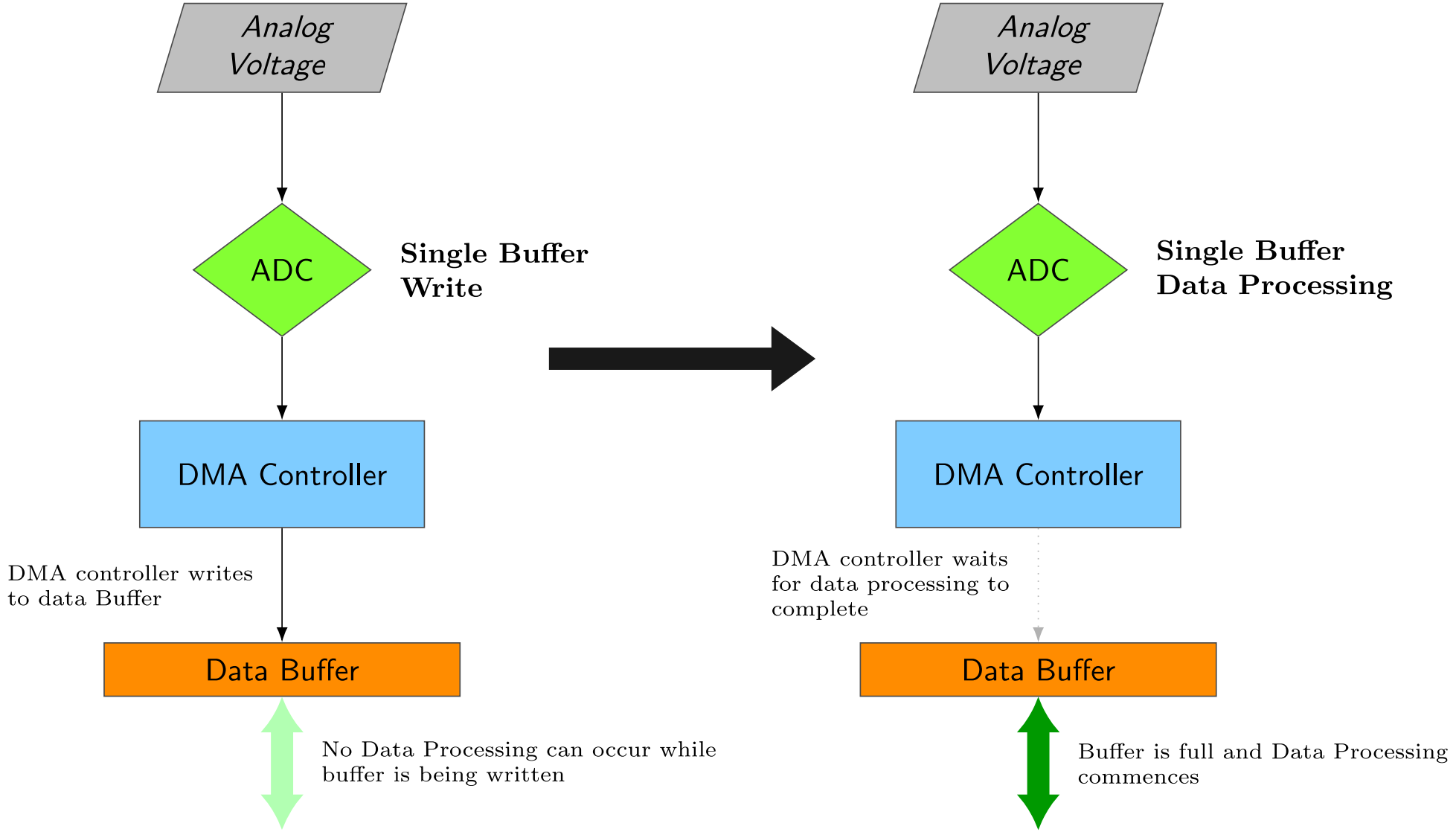




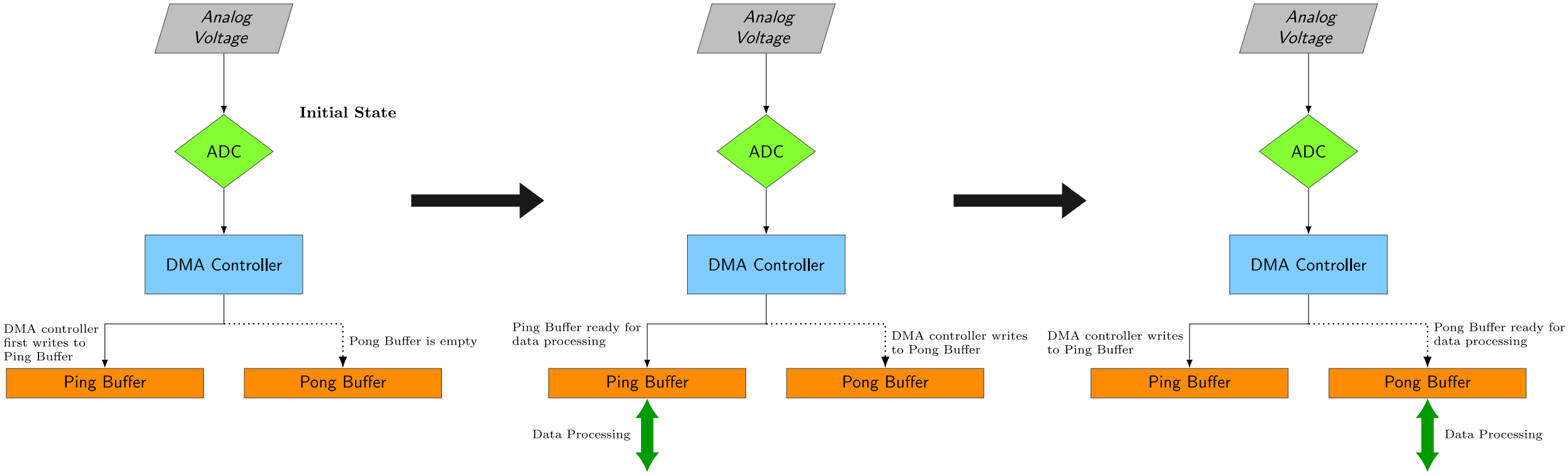
# Ping Pong Buffer



# FFT Code – Ping Pong Buffer (2/3)



# FFT Code – Ping Pong Buffer (3/3)




If this is not animated, I did not have the time to create it.

# Fourier Transforms & FFT Code (2/2)

# FFT Code - Setup (1/3)

```
EALLOW;
EPwm7Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group

EPwm7Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
EPwm7Regs.ETSEL.bit.SOCASEL = 0x2; // Select Event when counter equal to
PRD
EPwm7Regs.ETPS.bit.SOCAPRD = 0x1; // Generate pulse on 1st event
EPwm7Regs.TBCTR = 0x0; // Clear counter
EPwm7Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
EPwm7Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
EPwm7Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
EPwm7Regs.TBPRD = 5000; // Set Period to 0.1ms sample. Input clock is
50MHz.
EPwm7Regs.ETSEL.bit.SOCAEN = 1; // Enable SOC on A group
// Notice here that we are not setting CMPA or CMPB because we are not
using the PWM signal EPwm7Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
EPwm7Regs.TBCTL.bit.CTRMODE = 0x00; //unfreeze, and enter up count mode
EDIS;
```

# FFT Code - Setup (2/3)

```
EALLOW;
//write configurations for all ADCs ADCA, ADCB, ADCC, ADCD
AdcaRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4
AdcbRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4
AdccRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4
AdcdRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4

AdcSetMode(ADC_ADCA, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration settings
AdcSetMode(ADC_ADCB, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration settings
AdcSetMode(ADC_ADCC, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration settings
AdcSetMode(ADC_ADCD, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration settings

//Set pulse positions to late
AdcaRegs.ADCCTL1.bit.INTPULSEPOS = 1;
AdcbRegs.ADCCTL1.bit.INTPULSEPOS = 1;
AdccRegs.ADCCTL1.bit.INTPULSEPOS = 1;
AdcdRegs.ADCCTL1.bit.INTPULSEPOS = 1;   //power up the ADCs
AdcaRegs.ADCCTL1.bit.ADCPWDNZ = 1;
AdcbRegs.ADCCTL1.bit.ADCPWDNZ = 1;
AdccRegs.ADCCTL1.bit.ADCPWDNZ = 1;
AdcdRegs.ADCCTL1.bit.ADCPWDNZ = 1;      //delay for 1ms to allow ADC time to power up
DELAY_US(1000);

//ADCB
AdcbRegs.ADCSOC0CTL.bit.CHSEL = 4;      //SOC0 will convert Channel you choose Does not have to be B0, ADCIN4
AdcbRegs.ADCSOC0CTL.bit.ACQPS = 99;     //sample window is acqps + 1 SYSCLK cycles = 500ns
AdcbRegs.ADCSOC0CTL.bit.TRIGSEL = 0x11; // EPWM7 ADCSOCA or another trigger you choose will trigger SOC0
AdcbRegs.ADCINTSEL1N2.bit.INT1SEL = 0;  //set to last SOC that is converted and it will set INT1 flag ADCB1
AdcbRegs.ADCINTSEL1N2.bit.INT1E = 1;    //enable INT1 flag, originally = 1 (need to set to 1 for non-DMA)
AdcbRegs.ADCINTFLGCLR.bit.ADCINT1 = 1;  //make sure INT1 flag is cleared
AdcbRegs.ADCINTSEL1N2.bit.INT1CONT = 1; // Interrupt pulses regardless of flag state
EDIS;
// Initialize DMA
InitDma();
```

# FFT Code - Setup (3/3)

```
void InitDma(void)
{
    EALLOW;

    //--- Overall DMA setup
    DmaRegs.DMACTRL.bit.HARDRESET = 1;          // Reset entire DMA module
    asm(" NOP");                                // 1 cycle delay for HARDRESET to take effect

    DmaRegs.DEBUGCTRL.bit.FREE = 1;             // 1 = DMA unaffected by emulation halt
    DmaRegs.PRIORITYCTRL1.bit.CH1PRIORITY = 0;  // Not using CH1 Priority mode

    //--- Configure DMA channel 1 to read the ADC results
    DmaRegs.CH1.MODE.all = 0x8901; //good
    // 0x8901 = 1000 1001 0000 0001
    // CHINTE = interrupt enabled
    // CONTINUOUS = re-init after transfer complete
    // PERINTE = peripheral interrupt trigger enabled
    // Set Channel 1

    //--- Select DMA interrupt source                /******** TRIGGER SOURCE FOR EACH DMA CHANNEL (unlisted numbers are reserved) ********/
    DmaClaSrcSelRegs.DMACHSRCSEL1.bit.CH1 = 6;    // 0=none       6=ADCBINT1  12=ADCCINT2  18=ADCDINT3  32=XINT4      40=EPWM3SOCA  46=EPWM6SOCA  52=EPWM9SOCA   58=EPWM12SOCA  72=MREVTA    98=SD1FLT4    110=SPIRXDMAA
132=USBA_EPx_TX1
    DmaClaSrcSelRegs.DMACHSRCSEL1.bit.CH2 = 0;    // 1=ADCAINT1   7=ADCBINT2  13=ADCCINT3  19=ADCDINT4  33=XINT5      41=EPWM3SOCB  47=EPWM6SOCB  53=EPWM9SOCB   59=EPWM12SOCB  73=MXEVTB    99=SD2FLT1    111=SPITXDMAB
133=USBA_EPx_RX2
    DmaClaSrcSelRegs.DMACHSRCSEL1.bit.CH3 = 0;    // 2=ADCAINT2   8=ADCBINT3  14=ADCCINT4  20=ADCDEVT   36=EPWM1SOCA  42=EPWM4SOCA  48=EPWM7SOCA  54=EPWM10SOCA  68=TINT0       74=MREVTB   100=SD2FLT2    112=SPIRXDMAB
134=USBA_EPx_TX2
    DmaClaSrcSelRegs.DMACHSRCSEL1.bit.CH4 = 0;    // 3=ADCAINT3   9=ADCBINT4  15=ADCCEVT   29=XINT1     37=EPWM1SOCB  43=EPWM4SOCB  49=EPWM7SOCB  55=EPWM10SOCB  69=TINT1       95=SD1FLT1  101=SD2FLT3    113=SPITXDMAC
135=USBA_EPx_RX3
    DmaClaSrcSelRegs.DMACHSRCSEL2.bit.CH5 = 0;    // 4=ADCAINT4  10=ADCBEVT   16=ADCDINT1  30=XINT2     38=EPWM2SOCA  44=EPWM5SOCA  50=EPWM8SOCA  56=EPWM11SOCA  70=TINT2       96=SD1FLT2  102=SD2FLT4    114=SPIRXDMAC
136=USBA_EPx_TX3
    DmaClaSrcSelRegs.DMACHSRCSEL2.bit.CH6 = 0;    // 5=ADCAEVT   11=ADCCINT1  17=ADCDINT2  31=XINT3     39=EPWM2SOCB  45=EPWM5SOCB  51=EPWM8SOCB  57=EPWM11SOCB  71=MXEVTA      97=SD1FLT3  109=SPITXDMAA  131=USBA_EPx_RX1

    //--- DMA trigger source lock
    DmaClaSrcSelRegs.DMACHSRCSELLOCK.bit.DMACHSRCSEL1 = 0;              // Write a 1 to lock (cannot be cleared once set)
    DmaClaSrcSelRegs.DMACHSRCSELLOCK.bit.DMACHSRCSEL2 = 0;              // Write a 1 to lock (cannot be cleared once set)


    DmaRegs.CH1.BURST_SIZE.bit.BURSTSIZE = 0;                           // 0 means 1 word per burst
    DmaRegs.CH1.TRANSFER_SIZE = RFFT_SIZE-1;                            // RFFT_SIZE bursts per transfer

    DmaRegs.CH1.SRC_TRANSFER_STEP = 0;                                  // 0 means add 0 to pointer each burst in a transfer
    DmaRegs.CH1.SRC_ADDR_SHADOW = (Uint32)&AdcbResultRegs.ADCRESULT0;   // SRC start address

    DmaRegs.CH1.DST_TRANSFER_STEP = 1;                                  // 1 = add 1 to pointer each burst in a transfer
    DmaRegs.CH1.DST_ADDR_SHADOW = (Uint32)AdcPingBufRaw;                // DST start address Ping buffer


    DmaRegs.CH1.CONTROL.all = 0x0091; //good
    // 0x0091 = 1001 0001
    // ERRCLR = clear SYNCERR bit
    // PERINTCLR = clear periph event
    // RUN = Enable channel

    //--- Finish up
    EDIS;

    PieCtrlRegs.PIEIER7.bit.INTx1 = 1;  // Enable DINTCH1 in PIE group 7
    IER |= M_INT7;                      // Enable INT7 in IER to enable PIE group
}
```

**All that's really happening is that we use ePWM as a deterministic timer to trigger the ADC, which then feeds into the DMA until the buffer is populated!**

# FFT Code - Main (1/2)

```
int16_t i = 0;
float samplePeriod = 0.0001;

// Clear input buffers:
for(i=0; i < RFFT_SIZE; i++){
    fft_input[i] = 0.0f;
}

hnd_rfft->FFTSize   = RFFT_SIZE;
hnd_rfft->FFTStages = RFFT_STAGES;
hnd_rfft->InBuf     = &fft_input[0];    //Input buffer
hnd_rfft->OutBuf    = &test_output[0];  //Output buffer
hnd_rfft->MagBuf    = &pwrSpec[0];      //Magnitude buffer

hnd_rfft->CosSinBuf = &RFFTF32Coef[0];  //Twiddle factor buffer
RFFT_f32_sincostable(hnd_rfft);         //Calculate twiddle factor

for (i=0; i < RFFT_SIZE; i++){
      test_output[i] = 0;               //Clean up output buffer
}

for (i=0; i <= RFFT_SIZE/2; i++){
     pwrSpec[i] = 0;                    //Clean up magnitude buffer
}
```

# FFT Code - Main (2/2)

```
while(1)
{
    if ( (pingFFT == 1) || (pongFFT == 1) ) {
        if (pingFFT == 1) {
            pingFFT = 0;
            // Raw ADC data
            for(i=0; i<RFFT_SIZE; i++) {
                //--- Read the ADC results:
                fft_input[i] = AdcPingBufRaw[i]*3.0/4095.0;  // ping data
            }
        } else if (pongFFT == 1) {
            pongFFT = 0;
            // Raw ADC data
            for(i=0; i<RFFT_SIZE; i++) {
                //--- Read the ADC result:
                fft_input[i] = AdcPongBufRaw[i]*3.0/4095.0;  // pong data
            }
        }

        RFFT_f32(hnd_rfft);             // Send the fft data to be calculated
        RFFT_f32_mag_TMU0(hnd_rfft);    //Calculate magnitude

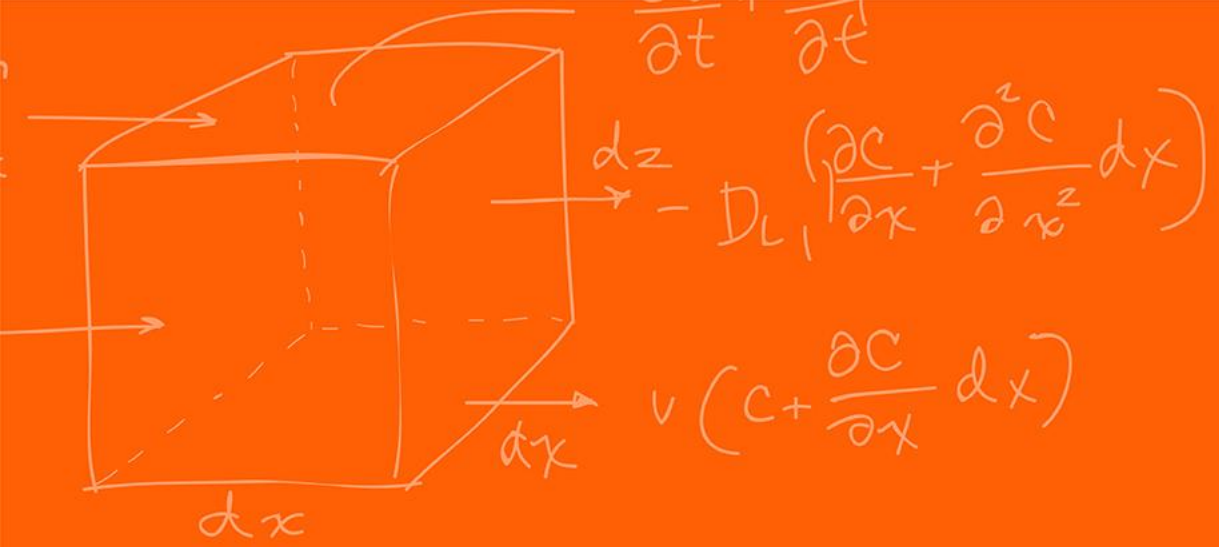
        maxpwr = 0;
        maxpwrindex = 0;

        // Ignore the bias ( low frequencies) and find the next max frequency
        for (i=5;i<(RFFT_SIZE/2);i++) {
            if (pwrSpec[i]>maxpwr) {
                maxpwr = pwrSpec[i];
                maxpwrindex = i;
            }
        }
    }
}
```

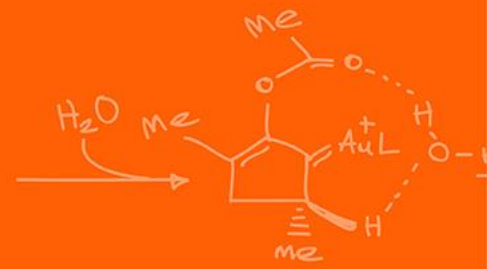
# FFT Code (5/5)

Given that we have the ePWM trigger the SOCA every 0.1ms and the FFT requires 1024 values, it thus takes 102.4ms for the data to fully be loaded into the buffers.

Thus, to compute the FFT's full pipeline it takes approximately 10Hz!

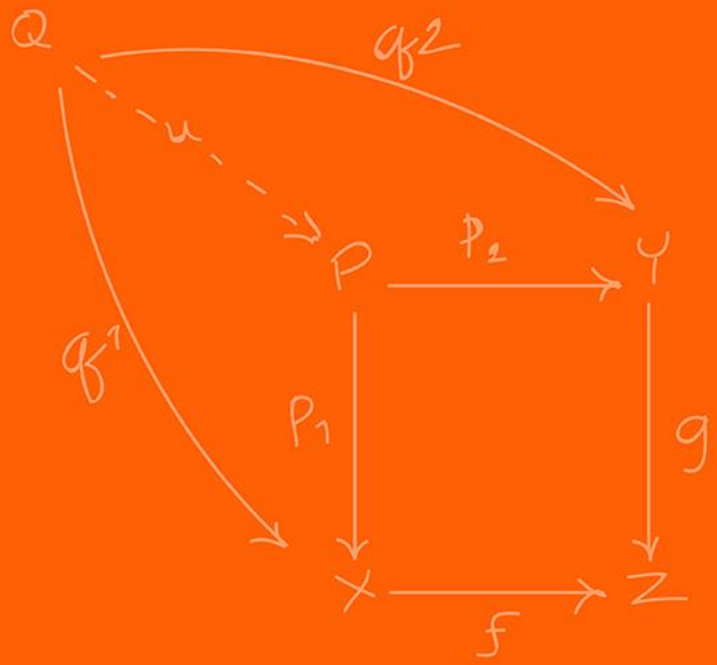








# FIR Filters



# FIR Filter (1/5)

Imagine you are building a robot that has an analog distance sensor, you feed it into your ADC and look at the output.

Instead of a smooth line telling you the wall is exactly 10cm away, the signal looks noisy.

Intuitively how would you smooth that data (think simple)?

You can just take the average of the last 5 points and the noise, spikes will attenuate!

# FIR Filter (2/5)

- Mathematically, at our current time step $n$, our filtered output $y[n]$ based on our noisy input's $x[n]$ looks like this:

$$y[n] = \frac{1}{5}x[n] + \frac{1}{5}x[n-1] + \frac{1}{5}x[n-2] + \frac{1}{5}x[n-3] + \frac{1}{5}x[n-4]$$

Without realizing it, you have just designed your first Finite Impulse Response (FIR) Filter!

# FIR Filter (3/5)

Note the structure, we have a set of **weights** (in the previous case, every weight was $\frac{1}{5}$ ) multiplied by a history of **states** (our current and past sensor readings).

We can generalize this idea into the fundamental equation of an FIR filter of order $M$.

$$y[n] = \sum_{k=0}^{M} b_k x[n-k]$$

Here, $b_k$ represents our filter coefficients (the “weights”).

# FIR Filter (4/5)

But why use a FIR filter?

You might wonder why we care so much about FIR filters specifically.

The answer is **Linear Phase**.

When a signal passes through a filter, different frequencies are often delayed by different amounts of time, which **distorts** the shape of the signal.

FIR filters can be designed to have *strict linear phase*, meaning every frequency is delayed by the **exact same** amount of time. The signal shape is perfectly preserved.

# FIR Filter (5/5)

- Our 5-point moving average is great, but what if we want to precisely block high-frequency motor noise but keep our low-frequency physical movements?

We need better coefficients ($b_k$)!

We use FIR filters and thus the **fir1** function to design those in MATLAB.

The Signal Processing Toolbox scales frequencies on a 0-1 scale relative to the Nyquist frequency. The Nyquist frequency is the absolute highest frequency your system can mathematically perceive, which is exactly half of your sampling rate.

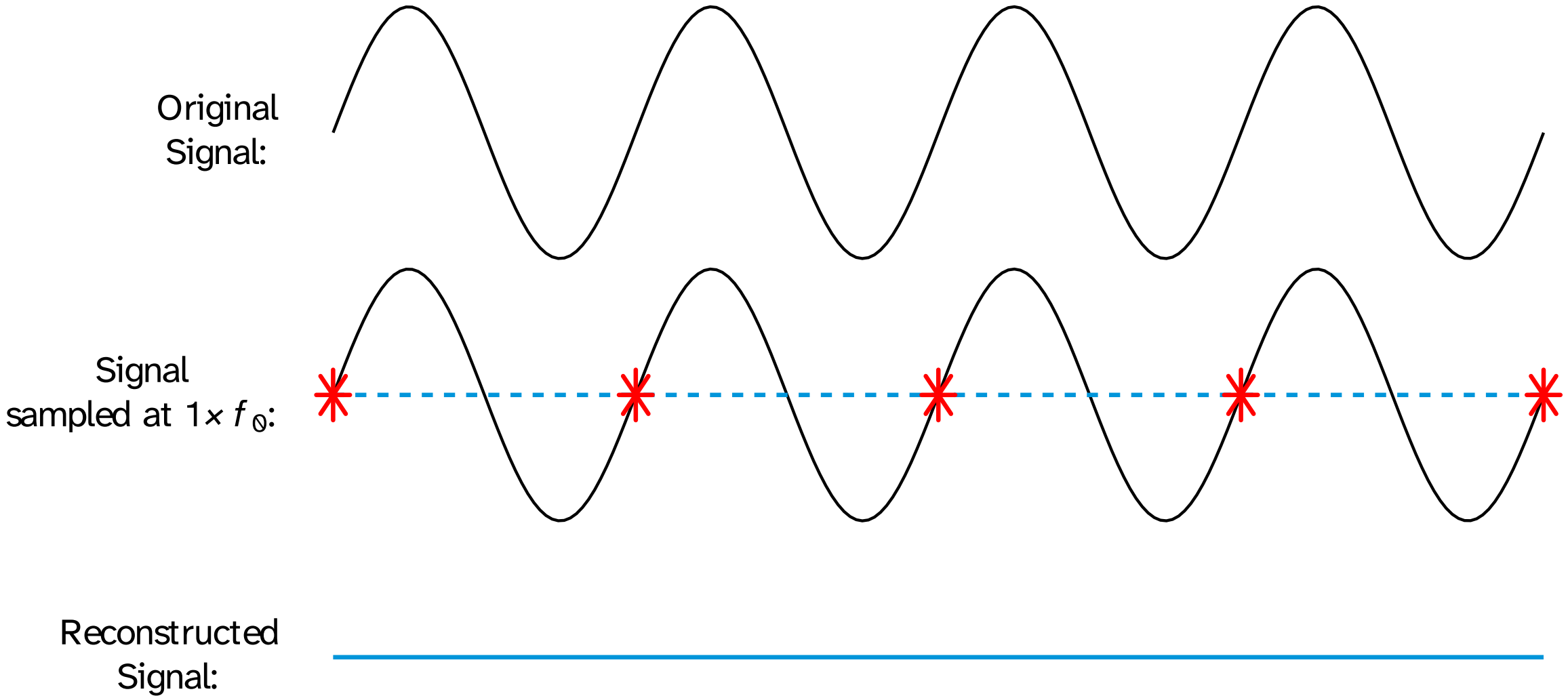
# FIR Filter – Nyquist frequency (1/5)

- The Nyquist frequency is the **MINIMUM** frequency you should be sampling your signal at.

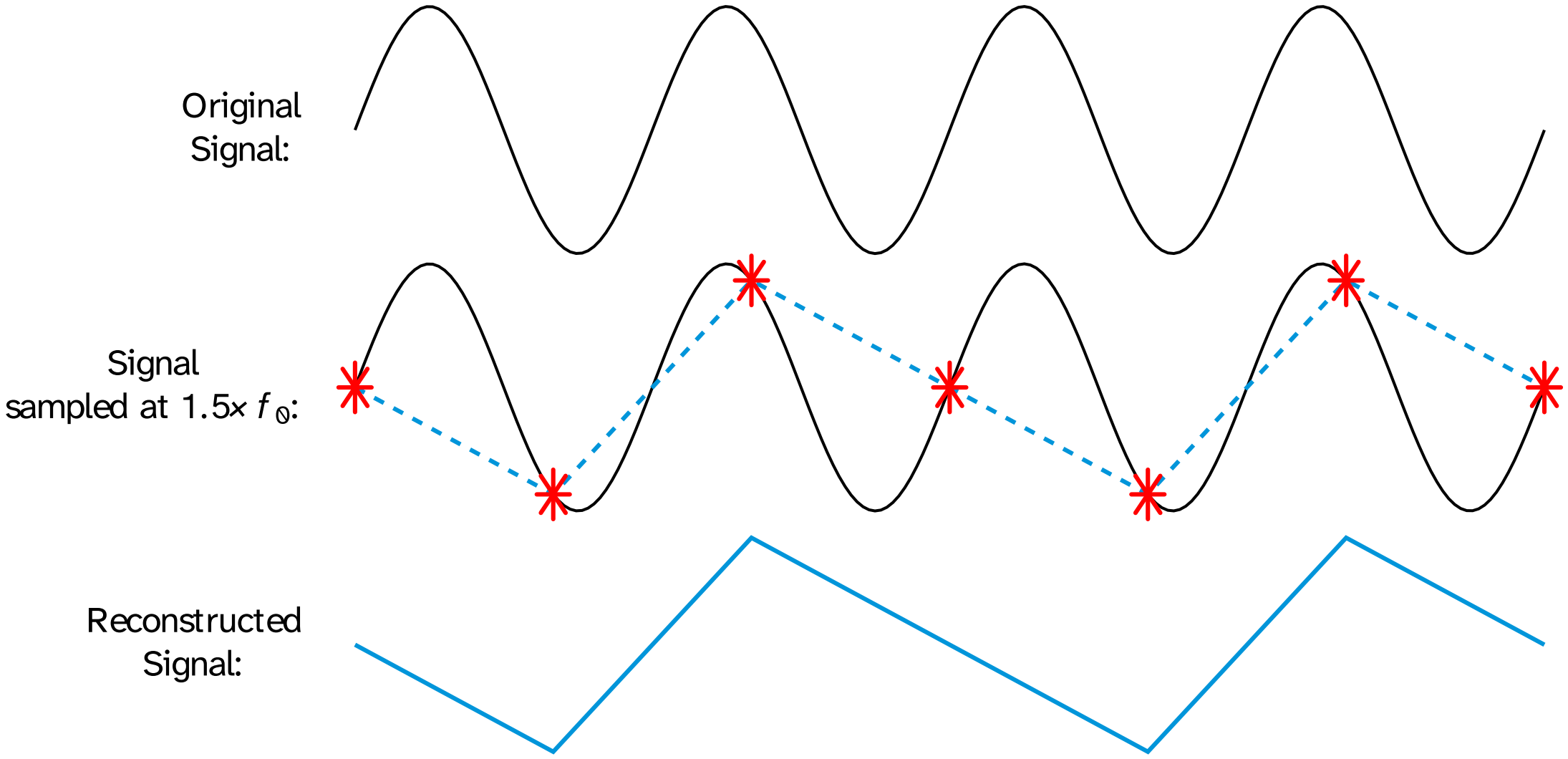
$$f_{nyquist} = 2 \times f_{highest\ analog\ frequency}$$

Otherwise, you will get into problems of getting an aliased output.

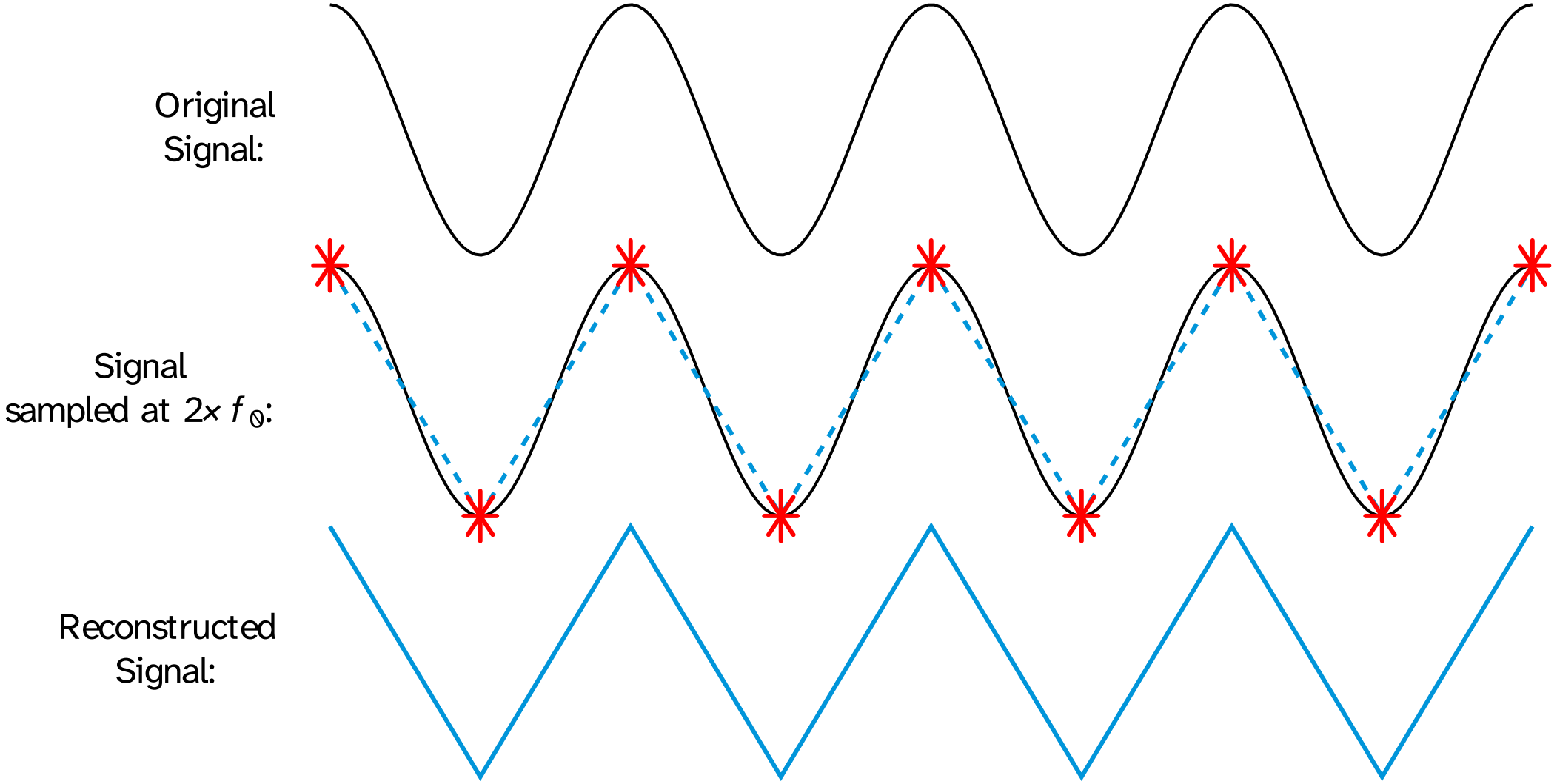
# FIR Filter – Nyquist frequency (2/5)



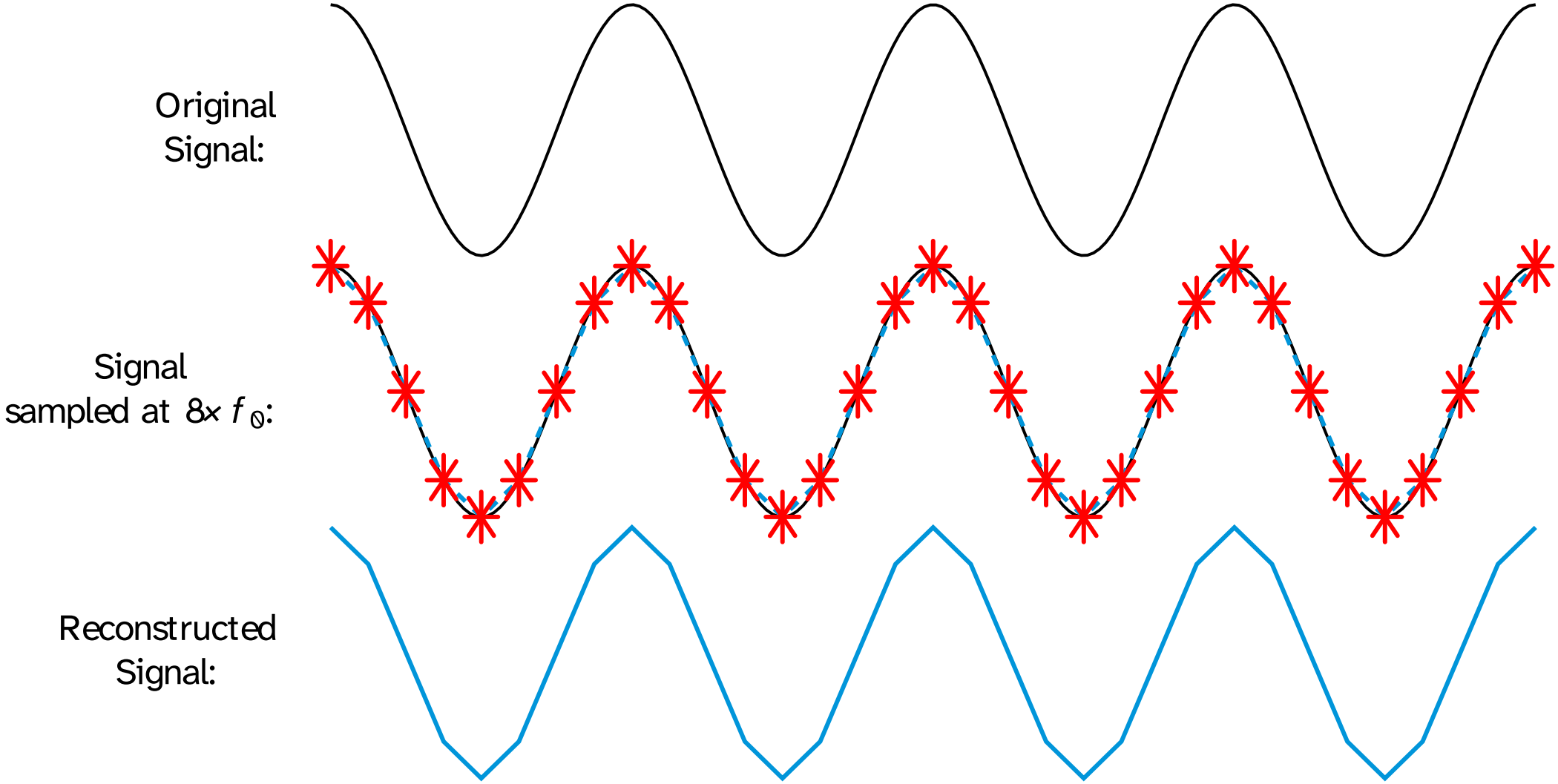
# FIR Filter – Nyquist frequency (3/5)



# FIR Filter – Nyquist frequency (4/5)



# FIR Filter – Nyquist frequency (5/5)



# FIR Code (1/7)

- Let's say your system is sampling at 1000 Hz; the cutoff frequency for the Nyquist limit is,

$$500 \text{ Hz},$$

If we want to do an FIR frequency design and do a 21st order filter with a low-pass cutoff of 100 Hz, we calculate the normalized frequency $W_n$

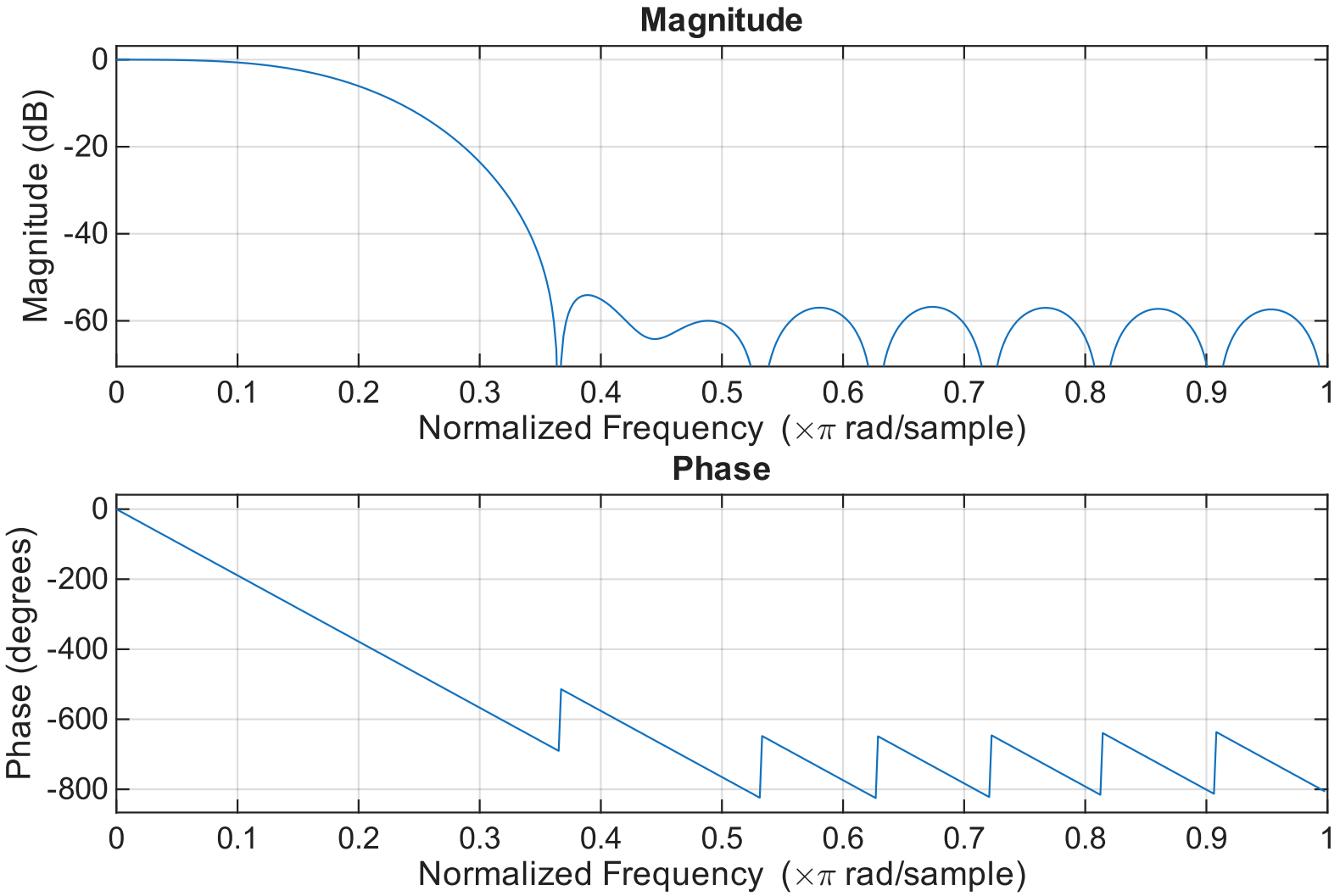
$$W_n = \frac{100 \text{ Hz}}{500 \text{ Hz}} = 0.2$$

# FIR Code (2/7)

- In MATLAB, generating the 22 weights ( a 21st has $M + 1$ coefficients) is incredibly simple!

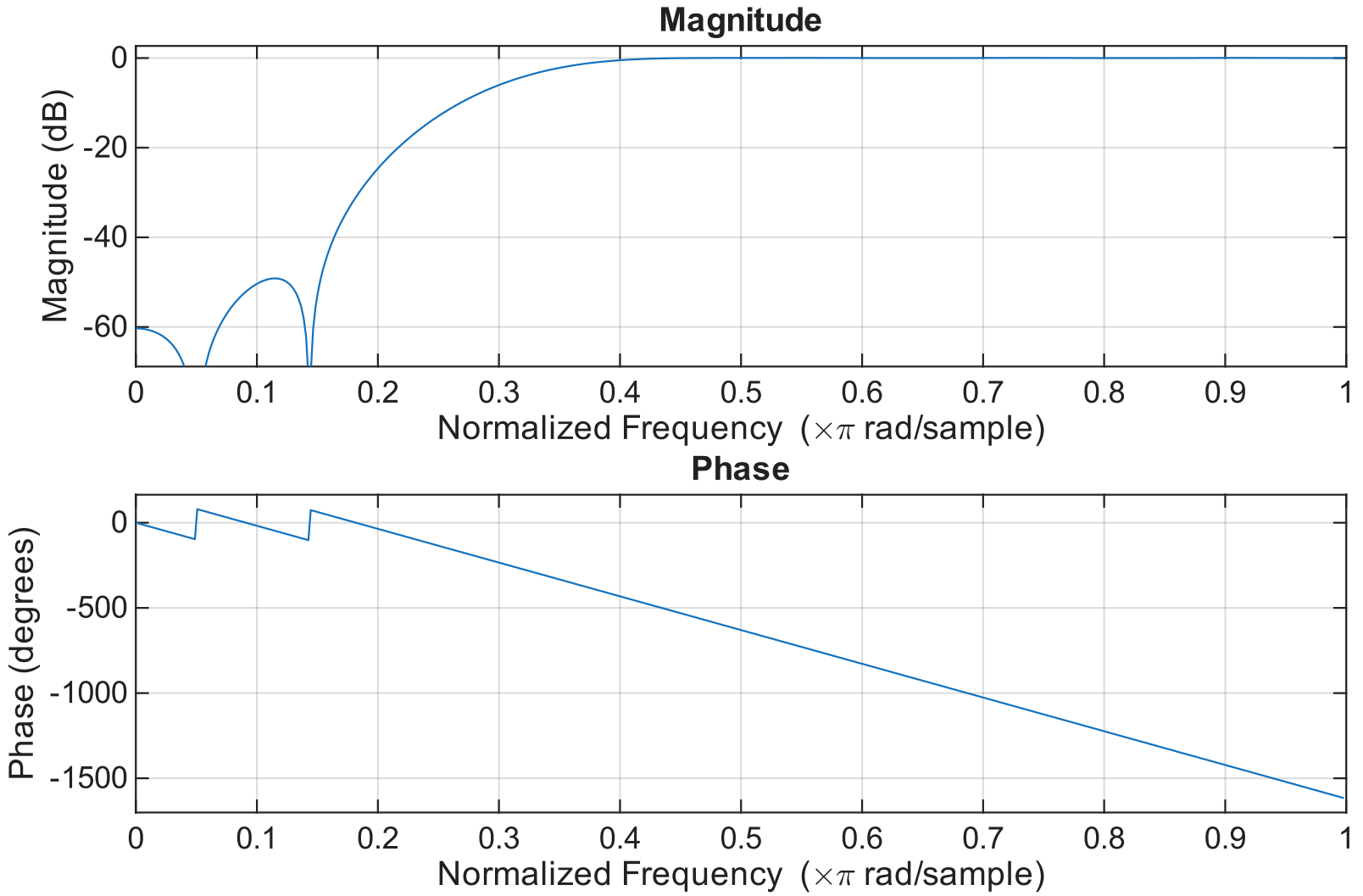
# FIR Code (3/7)

```
% 21st Order Low-Pass
    Filter
Order = 21;
Wn = 0.2;
b = fir1(Order, Wn, 'low');
freqz(b,1,512)
```



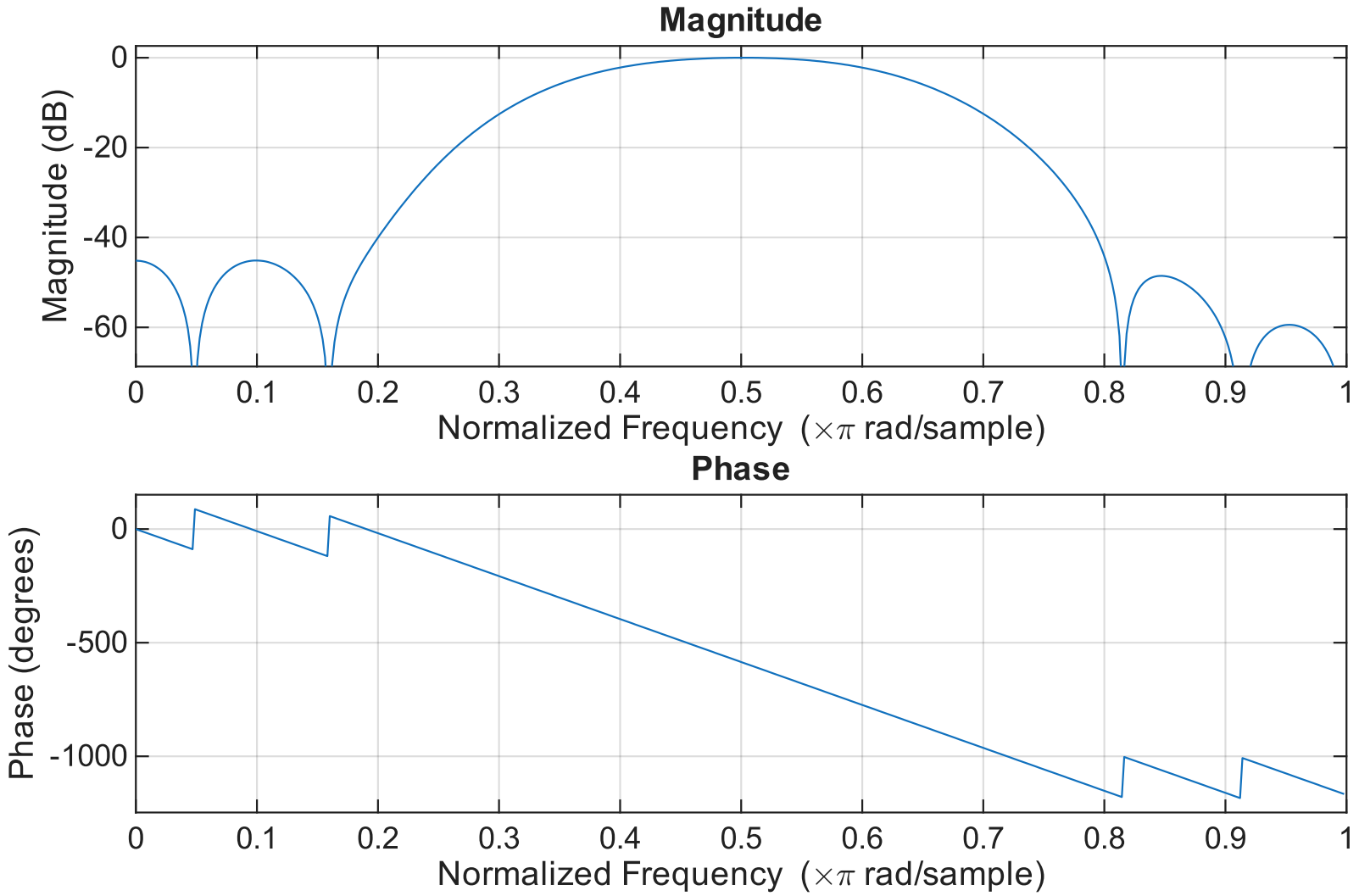
# FIR Code (4/7)

```
% 22st Order High-Pass
    Filter
Order = 22;
Wn = 0.3;
b = fir1(Order, Wn, 'high');
freqz(b,1,512)
```

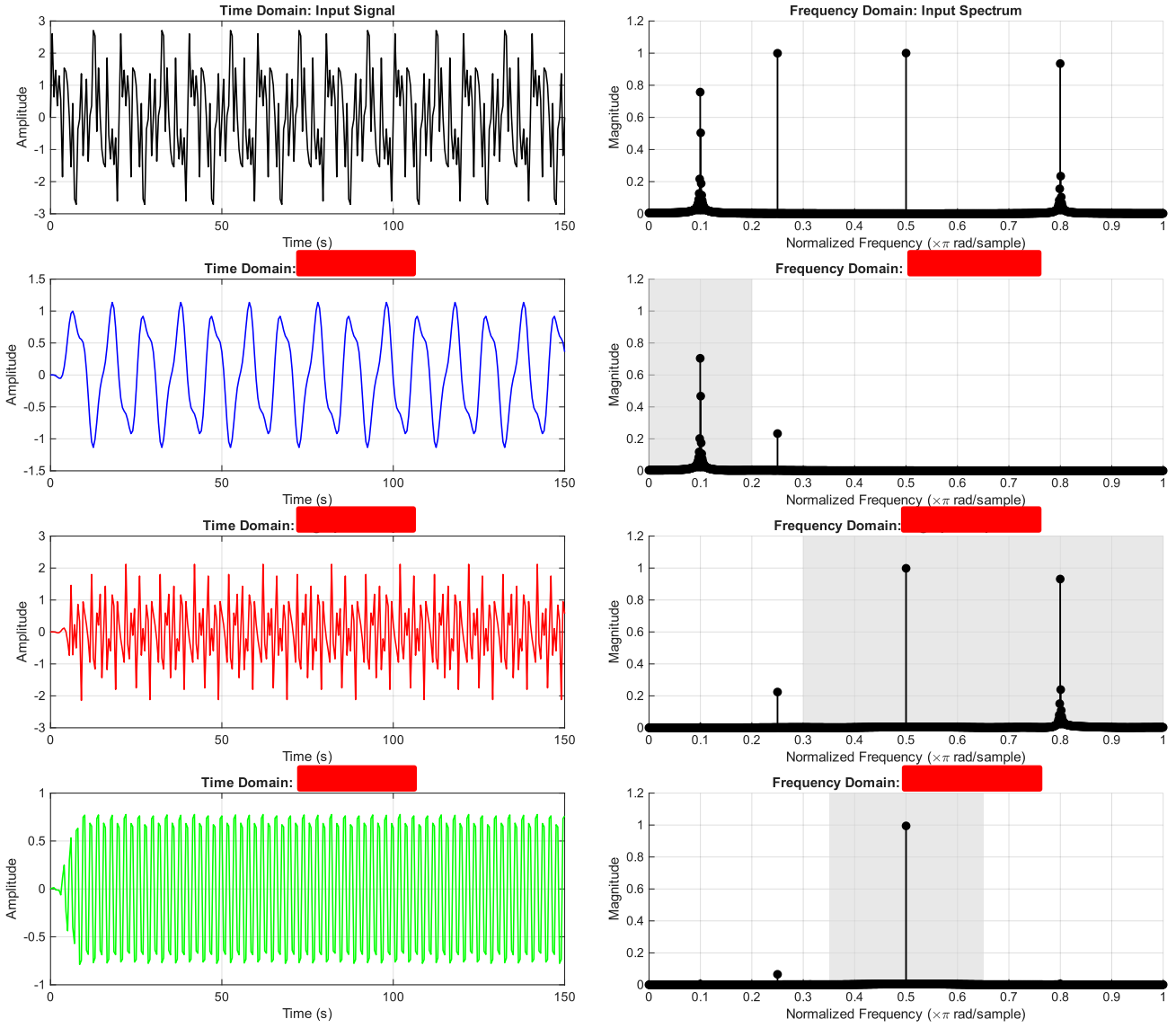


# FIR Code (5/7)

```
% 21st Order band-Pass
    Filter
Order = 21;
Wn = [0.35 0.65];
b = fir1(Order, Wn ,
    'bandpass');
freqz(b,1,512)
```



# FIR Code (6/7)


Time Domain: Input Signal
Amplitude
Time (s)
Frequency Domain: Input Spectrum
Magnitude
Normalized Frequency (×π rad/sample)
Time Domain:
Frequency Domain:
Time Domain:
Frequency Domain:
Time Domain:
Frequency Domain:

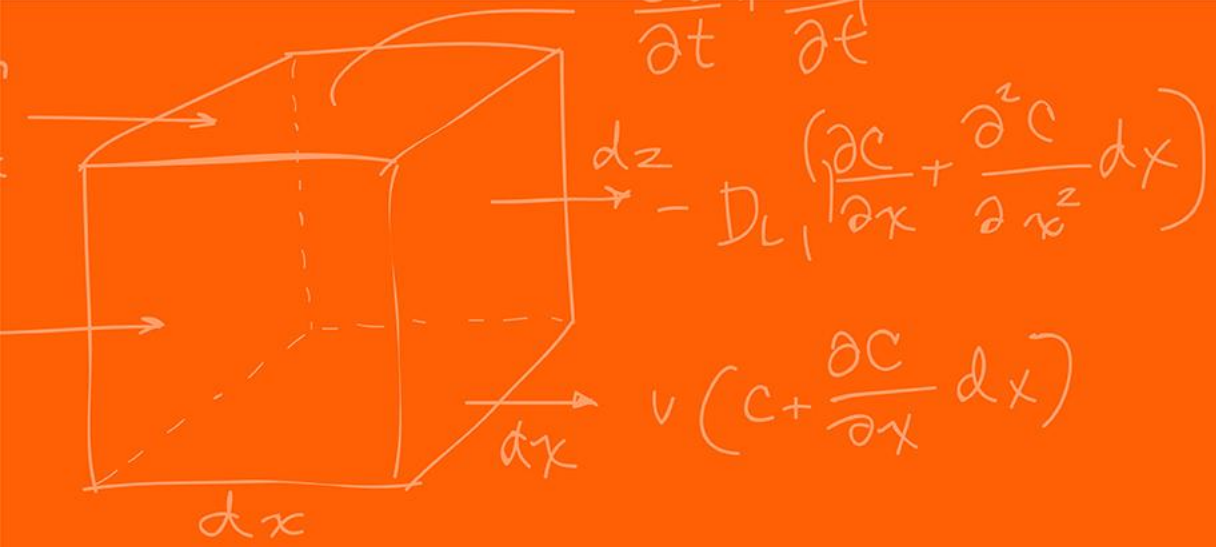
# FIR Code (7/7)

Now that we used MATLAB to generate the coefficients, we need to implement the actual filter in the code.

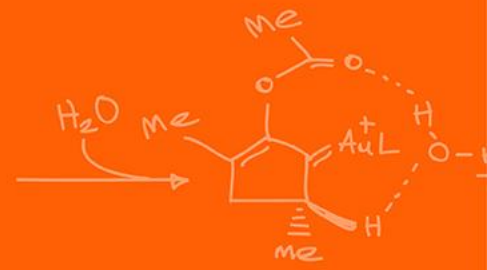
We do not need to save all the history when the robot is running only the running $M$ last states!

For a $M$-order filter, we only ever need the *current* reading and the $M$ *previous readings*!

# Questions? (2/2)









# The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

